

Histogrammar advanced tutorial

Histogrammar is a Python package that allows you to make histograms from numpy arrays, and pandas and spark dataframes. (There is also a scala backend for Histogrammar.)

This advanced tutorial shows how to:

- work with spark dataframes,
- make many histograms at ones, which is one of the nice features of histogrammar, and how to configure that. For example how to set bin specifications, or how to deal with a time-axis.

Enjoy!

```
In [1]: %%capture
# install histogrammar (if not installed yet)

!"{sys.executable}" -m pip install histogrammar
```

```
In [2]: import histogrammar as hg
```

```
In [3]: import pandas as pd
```

Data generation

Let's first load some data!

```
In [4]: # open a pandas dataframe for use below
from histogrammar import resources

df = pd.read_csv(resources.data("test.csv.gz"), parse_dates=["date"])
```

```
In [5]: df.head()
```

Out[5]:

		_id	address	age	balance	company	email	eyeColor	favoriteFruit	gender	guid	...	isActive	latitude	longitude	name
0	565ebe61ffa6f757b5029f29		109 Monument Walk, Vicksburg, California, 6247	16	\$1,333.24	DOGSPA	tamekajoyce@dogspa.com	grey	banana	female	c06a96e4-31e2-426e-aa5a-51e3c71991aa	...	False	18.821230	-131.721235	Tameka Joyce
1	565ebe616e073b23430d7052		602 Harkness Avenue, Tyhee, Arizona, 5853	57	\$3,777.44	APPLICA	rosettaflynn@applica.com	brown	banana	female	6bbcaf11-a6b9-4f00-9db1-84fc11161526	...	True	-82.134610	52.720886	Rosetta Flynn
2	565ebe6141fdb23a67dd7584		286 Fay Court, Forbestown, Pennsylvania, 6141	26	\$1,248.97	HOMETOWN	kathiemurray@hometown.com	blue	strawberry	female	b85ef702-39f4-42dd-a87f-ce97864c3551	...	True	62.383017	-85.223346	Kathie Murray
3	565ebe61c33636f6df907b06		617 Baycliff Terrace, Castleton, Oklahoma, 7714	63	\$1,049.77	ISIS	bennetthahn@isis.com	green	strawberry	male	b42f1df9-839f-4f39-b3de-7ea3ac177786	...	True	-26.913326	-67.825793	Bennett Hahn
4	565ebe61225511fd3288bb63		262 Scholes Street, Fillmore, Utah, 2905	74	\$2,942.64	FUTURITY	cottoncarson@futuraity.com	red	strawberry	male	14e518aa-2701-4397-94e7-72b9c27655ad	...	False	-2.777228	155.341981	Cotton Carson

5 rows × 21 columns

What about Spark DataFrames?

No problem! We can easily perform the same steps on a Spark DataFrame. One important thing to note there is that we need to include a jar file when we create our Spark session. This is used by spark to create the histograms using Histogrammar. The jar file will be automatically downloaded the first time you run this command.

In [6]:

```
# download histogrammar jar files if not already installed, used for histogramming of spark dataframe
try:
    from pyspark import __version__ as pyspark_version
    from pyspark.sql import SparkSession
    from pyspark.sql.functions import col
    pyspark_installed = True
except ImportError:
    print("pyspark needs to be installed for this example")
    pyspark_installed = False
```

pyspark needs to be installed for this example

In [7]:

```
# this is the jar file for spark 3.0
# for spark 2.X, in the jars string, for both jar files change "_2.12" into "_2.11".

if pyspark_installed:
```

```
scala = '2.12' if int(pyspark_version[0]) == 3 else '2.13'
hist_jar = f'io.github.histogrammar:histogrammar_{scala}:1.0.30'
hist_spark_jar = f'io.github.histogrammar:histogrammar-sparksql_{scala}:1.0.30'

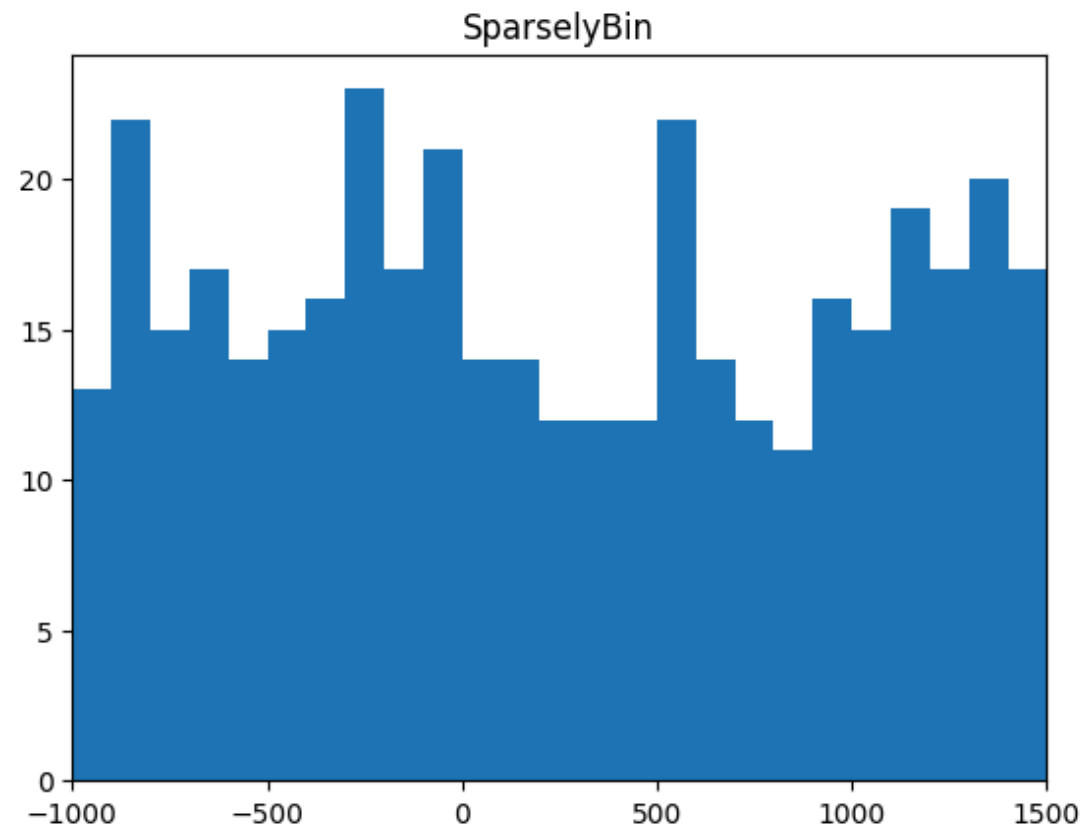
spark = SparkSession.builder.config(
    "spark.jars.packages", f'{hist_spark_jar},{hist_jar}'
).getOrCreate()

sdf = spark.createDataFrame(df)
```

Filling histograms with spark

Filling histograms with spark dataframes is just as simple as it is with pandas dataframes.

```
In [8]: # example: filling from a pandas dataframe
hist = hg.SparselyHistogram(binWidth=100, quantity='transaction')
hist.fill.numpy(df)
hist.plot.matplotlib();
```



```
In [9]: # for spark you will need this spark column function:
if pyspark_installed:
    from pyspark.sql.functions import col
```

Let's make the same histogram but from a spark dataframe. There are just two differences:

- When declaring a histogram, always set quantity to `col('columns_name')` instead of `'columns_name'`
- When filling the histogram from a dataframe, use the `fill.sparksql()` method instead of `fill.numpy()`.

```
In [10]: # example: filling from a pandas dataframe
if pyspark_installed:
    hist = hg.SparselyHistogram(binWidth=100, quantity=col('transaction'))
    hist.fill.sparksql(sdf)
    hist.plot.matplotlib()
```

Apart from these two differences, all functionality is the same between pandas and spark histograms!

Like pandas, we can also do directly from the dataframe:

```
In [11]: if pyspark_installed:
    h2 = sdf.hg_SparselyProfileErr(25, col('longitude'), col('age'))
    h2.plot.matplotlib()
```

```
In [12]: if pyspark_installed:
    h3 = sdf.hg_TwoDimensionallySparselyHistogram(25, col('longitude'), 10, col('latitude'))
    h3.plot.matplotlib()
```

All examples below also work with spark dataframes.

Making many histograms at once

Histogrammar has a nice method to make many histograms in one go. See [here](#).

By default automagical binning is applied to make the histograms.

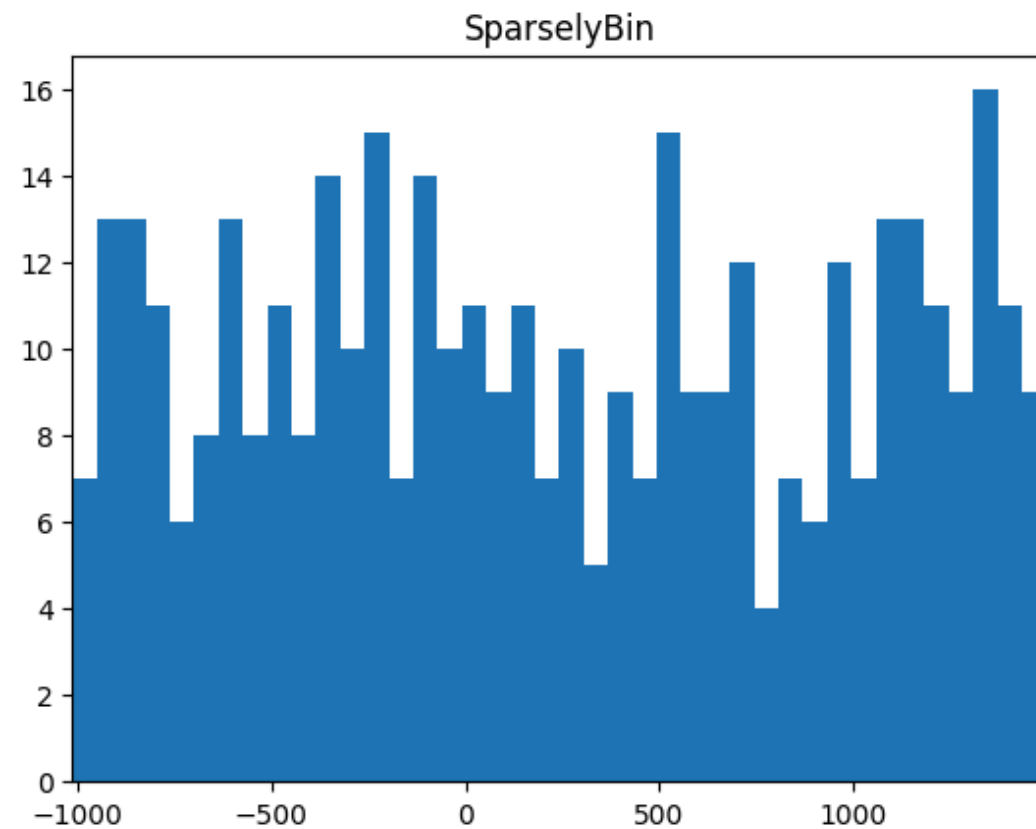
```
In [13]: hists = df.hg_make_histograms()
```

```
0%|          | 0/21 [00:00<?, ?it/s]
100%|██████████| 21/21 [00:00<00:00, 3032.13it/s]
```

```
In [14]: # histogrammar has made histograms of all features, using an automated binning.
hists.keys()
```

```
Out[14]: dict_keys(['_id', 'address', 'age', 'balance', 'company', 'currency', 'date', 'email', 'eyeColor', 'favoriteFruit', 'gender', 'guid', 'index', 'isActive', 'latitude', 'longitude', 'name', 'phone', 'registered', 'tags', 'transaction'])
```

```
In [15]: h = hists['transaction']
h.plot.matplotlib();
```



```
In [16]: # you can select which features you want to histogram with features=:
histo = df.hg_make_histograms(features = ['longitude', 'age', 'eyeColor'])
```

```
0%| | 0/3 [00:00<?, ?it/s]
100%| | 3/3 [00:00<00:00, 3851.52it/s]
```

```
In [17]: # you can also make multi-dimensional histograms
# here longitude is the first axis of each histogram.
histo = df.hg_make_histograms(features = ['longitude:age', 'longitude:age:eyeColor'])
```

```
0%| | 0/2 [00:00<?, ?it/s]
100%| | 2/2 [00:00<00:00, 172.64it/s]
```

Working with timestamps

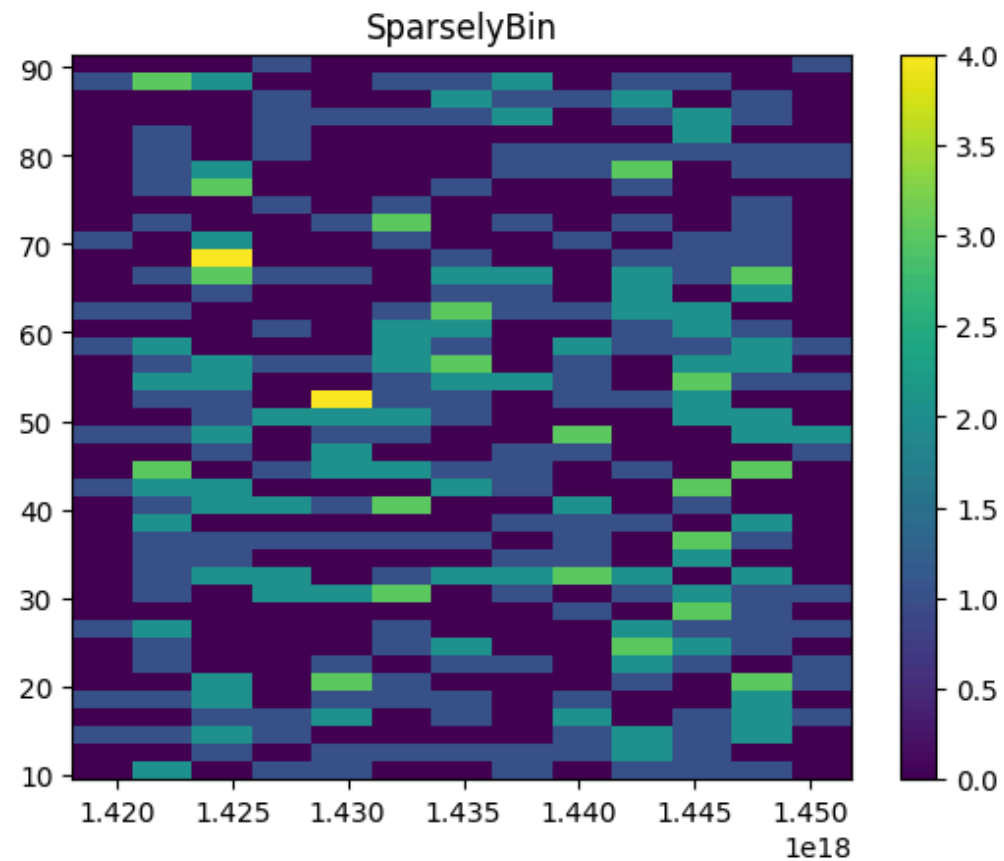
```
In [18]: # Working with a dedicated time axis, make histograms of each feature over time.
histo = df.hg_make_histograms(time_axis="date")
```

```
0%| | 0/20 [00:00<?, ?it/s]
100%| | 20/20 [00:00<00:00, 773.30it/s]
```

```
In [19]: histo.keys()
```

```
Out[19]: dict_keys(['date:_id', 'date:address', 'date:age', 'date:balance', 'date:company', 'date:currency', 'date:email', 'date:eyeColor', 'date:favoriteFruit', 'date:gender', 'date:uid', 'date:index', 'date:isActive', 'date:latitude', 'date:longitude', 'date:name', 'date:phone', 'date:registered', 'date:tags', 'date:transaction'])
```

```
In [20]: h2 = histo['date:age']
h2.plot.matplotlib();
```



Histogrammar does not support pandas' timestamps natively, but converts timestamps into nanoseconds since 1970-1-1.

```
In [21]: h2.bin_edges()
```

```
Out[21]: array([1.4180832e+18, 1.4206752e+18, 1.4232672e+18, 1.4258592e+18,
                1.4284512e+18, 1.4310432e+18, 1.4336352e+18, 1.4362272e+18,
                1.4388192e+18, 1.4414112e+18, 1.4440032e+18, 1.4465952e+18,
                1.4491872e+18, 1.4517792e+18])
```

The datatype shows the datetime though:

```
In [22]: h2.datatype
```

```
Out[22]: [numpy.datetime64, float]
```

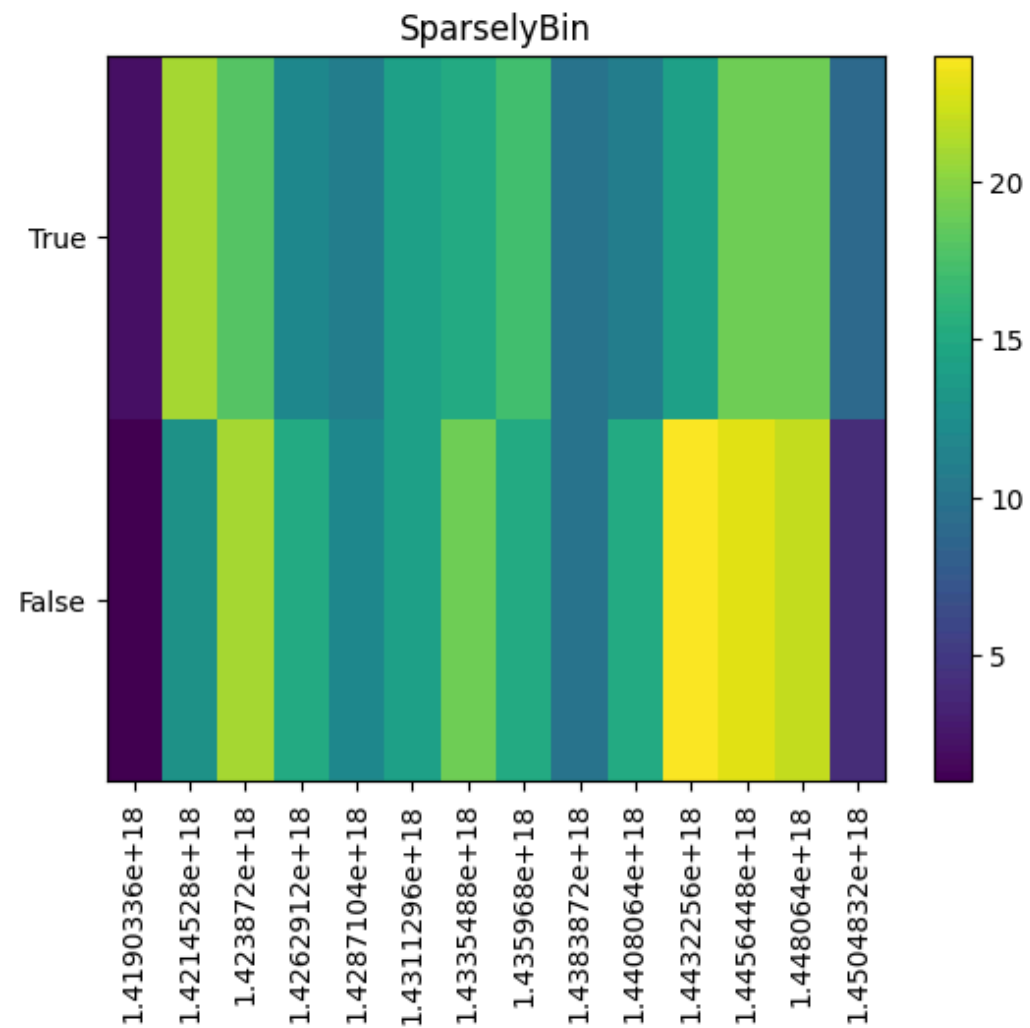
```
In [23]: # convert these back to timestamps with:
pd.Timestamp(h2.bin_edges()[0])
```

```
Out[23]: Timestamp('2014-12-09 00:00:00')
```

```
In [24]: # For the time axis, you can set the binning specifications with time_width and time_offset:
hists = df.hg_make_histograms(time_axis="date", time_width='28d', time_offset='2014-1-4', features=['date:isActive', 'date:age'])
```

```
0%| | 0/2 [00:00<?, ?it/s]
100%| | 2/2 [00:00<00:00, 821.61it/s]
```

```
In [25]: hists['date:isActive'].plot.matplotlib();
```

Setting binning specifications

```
In [26]: # histogram selections. Here 'date' is the first axis of each histogram.
features=[
    'date', 'latitude', 'longitude', 'age', 'eyeColor', 'favoriteFruit', 'transaction'
]

# Specify your own binning specifications for individual features or combinations thereof.
# This bin specification uses open-ended ("sparse") histograms; unspecified features get
# auto-binned. The time-axis binning, when specified here, needs to be in nanoseconds.
bin_specs={
    'longitude': {'binWidth': 10.0, 'origin': 0.0},
    'latitude': {'edges': [-100, -75, -25, 0, 25, 75, 100]},
    'age': {'num': 100, 'low': 0, 'high': 100},
    'transaction': {'centers': [-1000, -500, 0, 500, 1000, 1500]},
    'date': {'binWidth': pd.Timedelta('4w').value, 'origin': pd.Timestamp('2015-1-1').value}
}

# this binning specification is making:
# - a sparse histogram for: longitude
# - an irregular binned histogram for: latitude
# - a closed-range evenly spaced histogram for: age
```

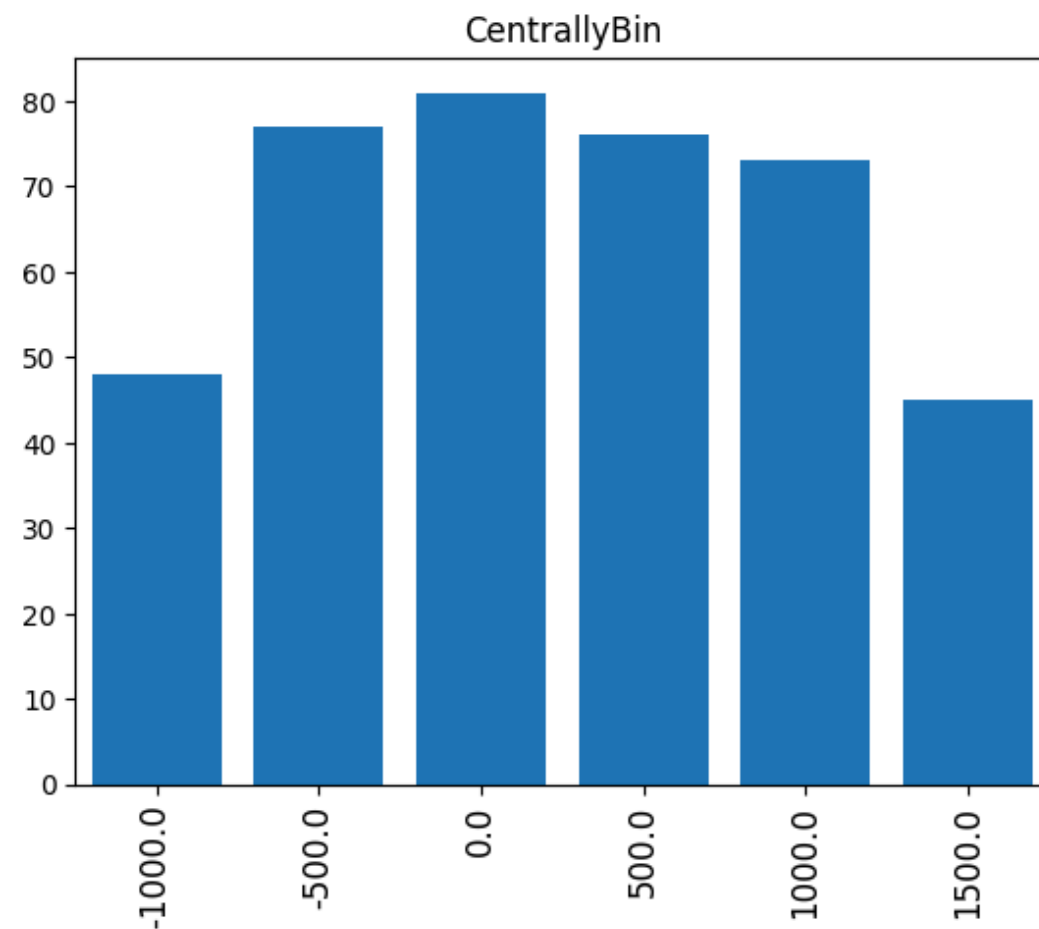
```
# - a histogram centered around bin centers for: transaction
hists = df.hg_make_histograms(features=features, bin_specs=bin_specs)
```

```
0%| | 0/7 [00:00<?, ?it/s]
100%| | 7/7 [00:00<00:00, 5343.06it/s]
```

```
In [27]: hists.keys()
```

```
Out[27]: dict_keys(['date', 'latitude', 'longitude', 'age', 'eyeColor', 'favoriteFruit', 'transaction'])
```

```
In [28]: hists['transaction'].plot.matplotlib();
```



```
In [29]: # all available bin specifications are (just examples):
```

```
bin_specs = {'x': {'bin_width': 1, 'bin_offset': 0},           # SparselyBin histogram
             'y': {'num': 10, 'low': 0.0, 'high': 2.0},       # Bin histogram
             'x:y': [{}, {'num': 5, 'low': 0.0, 'high': 1.0}], # SparselyBin vs Bin histograms
             'a': {'edges': [0, 2, 10, 11, 21, 101]},         # IrregularlyBin histogram
             'b': {'centers': [1, 6, 10.5, 16, 20, 100]},       # CentrallyBin histogram
             'c': {'max': True},                                # Maximize histogram
             'd': {'min': True},                                # Minimize histogram
             'e': {'sum': True},                                 # Sum histogram
             'z': {'deviate': True},                             # Deviate histogram
             'f': {'average': True},                             # Average histogram
             'a:f': [{}, {'edges': [0, 10, 101]}, {'average': True}], # IrregularlyBin vs Average histograms
             'g': {'thresholds': [0, 2, 10, 11, 21, 101]},     # Stack histogram
             'h': {'bag': True},                                 # Bag histogram
```



```
}

# to set binning specs for a specific 2d histogram, you can do this:
# if these are not provide, the 1d binning specifications are picked up for 'a:f'
bin_specs = {'a:f': [{'edges': [0, 10, 101]}, {'average': True}]}
```

```
In [30]: # For example
features = ['latitude:age', 'longitude:age', 'age', 'longitude']

bin_specs = {
    'latitude': {'binWidth': 25},
    'longitude': {'edges': [-100, -75, -25, 0, 25, 75, 100]},
    'age': {'deviate': True},
    'longitude:age': [{'binWidth': 25}, {'average': True}],
}

hists = df.hg_make_histograms(features=features, bin_specs=bin_specs)
```

```
0%|          | 0/4 [00:00<?, ?it/s]
100%|██████████| 4/4 [00:00<00:00, 3048.74it/s]
```

```
In [31]: h = hists['latitude:age']
h.bins
```

```
Out[31]: {np.int64(-4): <Deviate mean=43.4 variance=588.16>,
np.int64(-3): <Deviate mean=49.34920634920635 variance=523.0526581002771>,
np.int64(-2): <Deviate mean=44.60377358490566 variance=403.55998576005686>,
np.int64(-1): <Deviate mean=48.32692307692308 variance=443.29696745562137>,
np.int64(0): <Deviate mean=49.03508771929825 variance=574.3145583256387>,
np.int64(1): <Deviate mean=51.75 variance=495.62083333333334>,
np.int64(2): <Deviate mean=51.416666666666664 variance=388.30972222222224>,
np.int64(3): <Deviate mean=44.666666666666664 variance=521.3555555555557>}
```

```
In [32]: hists['longitude:age'].plot.matplotlib();
```

